Learning to Play the Field: What a Deep Q-Network Does and Does Not Learn in Round-Robin Iterated Prisoner's Dilemma

Kwame Asante-Boateng[1], **Yuki Hayashida**[2], **Léa Fournier-Dubois**[1]

[1]School of Computing, Calderbrook University, Toronto, Ontario, Canada
[2]Graduate School of Informatics, Shirakawa Institute of Science and Technology, Sendai, Japan
k.asanteboateng@calderbrook.ca, hayashida.yuki@inf.shirakawa-ist.ac.jp, lea.fournier-dubois@calderbrook.ca

Abstract

In a round-robin Iterated Prisoner's Dilemma tournament an agent does not face *an* opponent, it faces a *field*: a single policy must infer who it is playing from the history of the current match and best-respond, without ever being told the opponent's identity. We train a double deep Q-network for this setting over a 21-feature abstraction of the match history whose central ingredient is the opponent's cooperation rate *conditional* on our own previous move, and we evaluate it with a protocol borrowed from supervised learning rather than from the tournament literature: opponents are split into a training pool and a held-out set that never influences training or model selection, behavioural probes measure *how* an agent plays rather than how much it scores, and every number is reported over five independently trained networks. The protocol changes the conclusion. On aggregate tournament score the learned agent places a respectable fourth of twelve and beats the random baseline by $+0.617$ points per round ($p \approx 10^{-23}$); the held-out split shows that this hides a collapse in generalisation (2.70 against training opponents versus 2.09 against held-out ones, while a hand-written heuristic scores 2.70 and 2.99), and the probes localise the collapse to a single behaviour — escaping a mutual-defection spiral, where the network scores 0.14 against the heuristic's 0.99. A per-seed analysis further shows that the large variance of the learned agent is not noise but two distinct strategy basins that the random seed alone decides between. We argue that held-out opponents, probe suites and multi-seed reporting should be the default protocol for learned agents in repeated games.

Introduction

The Iterated Prisoner's Dilemma (IPD) has been the standard testbed for the study of cooperation since Axelrod's computer tournaments (Axelrod 1980a,b, 1984). It is the smallest repeated game in which the tension between individual and collective interest is non-trivial: two players simultaneously choose to cooperate (C) or defect (D), defection is dominant in a single round, and yet mutual cooperation is what pays over a long match. The literature that grew from Axelrod's tournaments established both the canonical evaluation protocol — a round robin over a field of strategies — and the canonical strategy set: tit-for-tat, grim trigger, Pavlov, generous tit-for-tat, prober (Nowak and Sigmund 1992, 1993).

That protocol has a property which is easy to overlook and which motivates this paper. In a round robin an agent does not play against *an* opponent; it plays against a *field*. A single fixed policy must handle an unconditional cooperator, an unconditional defector, a strict reciprocator and a randomiser, and the only signal that distinguishes them is the history of the match currently being played. The optimal behaviour against each of these is different — exploit, punish, reciprocate, exploit — so a competent agent in this setting is necessarily performing online opponent identification followed by best response. This is a materially harder learning problem than the one usually studied, in which a learner is trained and evaluated against a single fixed adversary.

Reinforcement learning is routinely proposed for repeated social dilemmas, and deep RL increasingly so (Leibo et al. 2017; Lerer and Peysakhovich 2017; Foerster et al. 2018). But reported results in this area are typically aggregate scores against a fixed pool of opponents, and an aggregate score against the pool one trained on cannot distinguish two very different situations: an agent that has *learned to play the game*, and an agent that has *memorised the pool*. The distinction matters exactly in the tournament setting, where the field is not known in advance — Axelrod's second tournament (Axelrod 1980b) was already a demonstration that the field changes and that what scores well changes with it.

This paper is a careful small-scale study of that distinction. We build a complete, dependency-free IPD tournament framework, a hand-written heuristic baseline of the kind a game theorist would produce in an afternoon, and a double deep Q-network (Mnih et al. 2015; van Hasselt, Guez, and Silver 2016) trained by self-play and play against a pool of classic strategies. We then evaluate all three with an instrument set that is standard in supervised learning and rare in this literature: a held-out opponent split, a suite of behavioural probes, component ablations and multi-seed reporting with confidence intervals.

Contributions.

1. **A state abstraction for playing a field.** We encode the unbounded match history into 21 features, of which the decisive two are the opponent's cooperation rate conditional on our own previous C and on our own previous D. These two numbers are what allow a *single* network to separate a reciprocator from an unconditional cooperator, and therefore what makes one policy able to play a whole field rather than one opponent.

2. **An evaluation protocol for learned agents in repeated games.** Training opponents are disjoint from held-out opponents, held-out opponents never influence model selection, behavioural probes report interpretable strategy descriptions rather than scores, and every number is reported over five independently trained networks with Welch intervals.
3. **A negative-but-informative result, localised.** The learned agent beats the random baseline decisively ($+0.617$ per round, 95% CI $[+0.559, +0.675]$, $p = 1.5 \times 10^{-23}$) but loses to the hand-written heuristic (-0.110, 95% CI $[-0.162, -0.057]$, $p = 3.1 \times 10^{-4}$) and to plain tit-for-tat. The deficit is *entirely* generalisation — the network matches the heuristic on the opponents it trained against (2.70 vs 2.70) and collapses on the ones it has not (2.09 vs 2.99) — and the probes localise it to one behaviour: it never escapes a mutual-defection spiral (recovery 0.14 vs 0.99).
4. **Two basins, decided by the seed.** A per-seed analysis shows that the learned agent's large variance is not within-run noise: two of five seeds learn to probe and exploit unconditional cooperators (4.96 and 4.98 per round against `always_cooperate`, at the cost of scoring 1.02 against a grim trigger), while three learn never to defect first (3.00 against both). Both groups score within 0.08 of each other against the training pool, so a single-seed report of this agent is a coin flip between two qualitatively different strategies.

The methodological point is the transferable one. The aggregate tournament table alone would have reported a respectable fourth place out of twelve and hidden every one of these findings.

Related Work

The IPD and tournament play. Axelrod's two tournaments (Axelrod 1980a,b) and the subsequent synthesis (Axelrod and Hamilton 1981; Axelrod 1984) established tit-for-tat's success and the design principles — be nice, retaliate, forgive, be clear — that our behavioural probes are a direct attempt to measure. Later work qualified the picture: win-stay-lose-shift (Pavlov) outperforms tit-for-tat in noisy and evolutionary settings (Nowak and Sigmund 1993, 1992), generous tit-for-tat is the natural answer to implementation noise (Molander 1985; Wu and Axelrod 1995), and strategies with longer memory or explicit opponent recognition do better still (Lee, Harper, and Fryer 2015; Knight et al. 2024). Press and Dyson (2012) showed that zero-determinant strategies can unilaterally set a linear relation between the two players' scores, and the evolutionary stability of such extortion has been studied extensively (Stewart and Plotkin 2013; Hilbe, Nowak, and Sigmund 2013; Wang et al. 2016b). Modern reimplementations of the tournament setting, above all the Axelrod-Python library (Knight et al. 2016; Glynatsi and Knight 2021; Knight et al. 2025), have made large-field round robins reproducible; our framework is much smaller in scope but adopts the same convention of a shared strategy interface and a scripted round robin. Retrospectives on two decades of IPD competitions (Chong et al. 2007; Hingston and Kendall 2004; Hingston et al. 2007) document the same instability that motivates our held-out split: what wins depends on who else entered.

Learning in repeated games. Q-learning (Watkins and Dayan 1992) in matrix and repeated games has a long history (Sandholm and Crites 1996; Claus and Boutilier 1998; Littman 1994), and its central difficulty — the environment is non-stationary because the opponent is also adapting or is drawn from a distribution — is the reason our training procedure samples a pool rather than fixing one adversary (Buşoniu, Babuška, and De Schutter 2008; Hernandez-Leal, Kartal, and Taylor 2019; Bowling and Veloso 2002). Evolutionary and genetic approaches to IPD strategy discovery (Axelrod 1987; Fogel 1993; Ashlock and Kim 2008) attack the same problem from the representation side. Harper et al. (2017) apply reinforcement learning within the Axelrod-Python field and, like us, find that learned strategies are strong within the field they were tuned on. The folk theorems (Fudenberg and Maskin 1986) and the finite-horizon reputation results of Kreps et al. (1982) explain why our finite matches admit end-game defection, which both of our engineered agents exploit and which Selten and Stoecker (1986) document empirically in human play.

Deep Q-learning. We use the standard DQN recipe: experience replay (Lin 1992), a periodically synchronised target network (Mnih et al. 2013, 2015), the Huber loss (Huber 1964), Adam (Kingma and Ba 2015), and double Q-learning (van Hasselt 2010; van Hasselt, Guez, and Silver 2016) to remove the overestimation bias of the max operator (Thrun and Schwartz 1993). The overestimation argument deserves a game-specific remark that we have not seen made explicitly for the IPD: because defection is precisely the action with the largest *immediate* payoff, an optimistically biased target does not merely slow convergence, it biases the *learned strategy* toward defection. Double Q-learning is therefore a strategy-level choice here, not only an optimisation-level one — which is what made it worth an ablation. We deliberately do not use the further refinements of the Rainbow line (Schaul et al. 2016; Wang et al. 2016a; Hessel et al. 2018); the recurrent variant (Hausknecht and Stone 2015) is, however, the most obvious remedy for the failure we diagnose.

Multi-agent deep RL in social dilemmas. Sequential social dilemmas (Leibo et al. 2017) generalise the matrix game to temporally extended settings, and a family of methods now exists for sustaining cooperation in them: approximate tit-for-tat with learned models (Lerer and Peysakhovich 2017), opponent-aware learning (Foerster et al. 2018; Letcher et al. 2019), intrinsic rewards for influence, reciprocity or inequity aversion (Jaques et al. 2019; Eccles et al. 2019; Hughes et al. 2018; Peysakhovich and Lerer 2018), and opponent modelling networks (He et al. 2016). Population-based and open-ended training (Jaderberg et al. 2017; Lanctot et al. 2017; Balduzzi et al. 2019; Heinrich and Silver 2016) addresses the pool-composition problem that our ablation identifies as the only component with a clear effect. We position this paper not as a scaling result but

as a diagnosis: our contribution is to show, in a setting small enough that every claim is exactly reproducible, precisely which behaviour a standard deep Q-learner fails to acquire and precisely which measurement instrument reveals it.

Generalisation and reproducibility in RL. Our protocol is directly inspired by the generalisation literature in deep RL (Cobbe et al. 2019, 2020; Zhang et al. 2018; Kirk et al. 2023), which established that a policy evaluated on the levels it trained on tells you almost nothing about the policy. We treat opponents in a repeated game exactly as that literature treats levels. Likewise, the finding that our results split into two seed-determined basins is an instance of the variance problem documented by Henderson et al. (2018), Colas, Sigaud, and Oudeyer (2019) and Agarwal et al. (2021); our contribution is to show that in the IPD the variance is not a smear around a mean but a bimodal distribution over qualitatively different strategies, which is a stronger reason to distrust single-seed reports. Statistics follow Welch (1947) and Efron (1979).

The Game and Its Rules

Payoffs. Two players simultaneously choose C or D each round. With $T > R > P > S$ and $2R > T + S$, the per-round payoff to a player is R for mutual cooperation, P for mutual defection, T for defecting against a cooperator and S for cooperating against a defector. We use the classic values T=5, R=3, P=1, S=0, which our engine exposes as a parameter. D strictly dominates C in a one-shot game, while $2R > T + S$ makes sustained mutual cooperation better for both players than alternating exploitation.

Matches and tournaments. A *match* is a fixed number of rounds between two agents. A *tournament* is a round robin in which every agent plays one match against every other agent; agents are ranked by total score, not by head-to-head wins. An agent sees the complete history of the current match, but never the opponent's move for the round it is currently deciding. Agent instances persist for a whole tournament, so an agent *may* carry knowledge from one match to the next, while every tournament is played by a fresh set of agents, which keeps tournaments independent samples for the purpose of statistics.

Two consequences drive the rest of the paper. First, matches are finite and their length is known, so backward induction favours defection in the last rounds; both of our engineered agents exploit this, and the learned agent must discover it. Second, the state of the game is the unbounded history, so any function approximator needs a state abstraction — the subject of the next section.

The field. The twelve agents in our tournament are the learned agent, the two baselines (random and heuristic) and the nine classic strategies of Table 1, each of which is diagnostic of a different capability. `always_cooperate` tests whether an agent will exploit; `always_defect` whether it will punish; `tit_for_tat` and `suspicious_tit_for_tat` whether it can establish and re-establish cooperation; `grim_trigger` whether it understands that some defections are unrecoverable; `pavlov` and `generous_tit_for_tat` whether it can cope with an opponent that forgives; `tit_for_two_tats` and `prober` whether it can resist being exploited by an opponent that tests it first.

Strategy	Behaviour
`always_cooperate`	always C; maximally exploitable
`always_defect`	always D; safe, unrewarding
`tit_for_tat`	C, then copy opponent's last move
`suspicious_tit_for_tat`	tit-for-tat opening with D
`tit_for_two_tats`*	retaliate only after two Ds
`grim_trigger`	C until first D, then D forever
`pavlov`	win-stay, lose-shift
`generous_tit_for_tat`	tit-for-tat, forgives w.p. 0.3
`prober`*	DCC, exploit if never punished
`random`	C with probability 0.5
`heuristic`*	classify opponent, best-respond
`dqn`	the learned agent

Table 1: The twelve-agent tournament field. A star marks the three opponents held out of the learned agent's training pool and of its model selection.

Approach

Architecture

The implementation is split into four layers that do not reach into each other: the game engine (moves, payoffs, matches, round robins) knows nothing about agents; agents implement a four-method interface (`reset` before a match, `act` each round, `observe` after each round, `finalize` after the match) and know nothing about how experiments are run; the learning package owns the state encoder, network, replay buffer and training loop; and the experiment harness runs tournaments and aggregates statistics. The whole system is pure Python plus NumPy.

This is not merely tidiness. Because the engine is ignorant of agent internals, the random baseline, the hand-written heuristic and the learned agent are measured on exactly identical terms — the same match loop, the same information, the same statistics — which is what makes the comparison in the results section a comparison of strategies rather than of harnesses. The Q-network is small enough that a hand-written NumPy multilayer perceptron is faster than a deep learning framework and, more importantly, makes a training run bit-for-bit reproducible from a single integer seed.

The Heuristic Baseline

Beating a random player is easy. The interesting reference point is a rule set that a game theorist would write down after an afternoon of thinking, so we implemented one:

1. **Probe.** Open cooperatively but defect once early (round 3). This is the only way to learn whether an opponent retaliates: a pure tit-for-tat can never tell an unconditional cooperator from a reciprocator, because it never gives either of them anything to react to.
2. **Classify.** After a warm-up of 8 rounds, label the opponent from its overall cooperation rate and, crucially,

its cooperation rate *conditional* on our own previous move: a *pushover* keeps cooperating even after we defect ($\Pr(\mathsf{C} \mid \text{our } \mathsf{D}) \geq 0.75$), a *reciprocator* cooperates markedly more after our C than after our D (a gap of at least 0.25), a *defector* cooperates less than 15% of the time overall, and anything else is *unresponsive*.

3. **Best-respond.** Defect against pushovers, defectors and unresponsive opponents. Against reciprocators play tit-for-tat, but cooperate unconditionally whenever the last two rounds contained no round of mutual cooperation.
4. **Defect at the very end,** in the final round, where reciprocation can no longer be punished.

Rule 3's second clause is the one to watch. It is an *olive branch*: a stretch without mutual cooperation covers both failure modes of plain tit-for-tat — an echo of mutual defection, and the CDCD alternation two mirroring agents fall into after a single defection — and the agent responds by cooperating until a round of mutual cooperation actually occurs. Crucially the olive branch is *unconditional*: it is not something that has to pay off in expectation, it just happens. This single rule is what the learned agent fails to discover, and it accounts for most of the gap between the two.

State Encoding

The history grows without bound, so the learner needs a fixed-length abstraction. We use 21 features, all in $[0, 1]$ so that no input normalisation layer is needed:

- **Sliding window (12 features).** One-hot encodings of both players' moves in each of the last three rounds. A one-round window already represents every memory-one strategy exactly, so three rounds comfortably cover the reactive strategies in the field.
- **Running statistics (9 features).** Our cooperation rate, the opponent's cooperation rate, the mutual cooperation rate, the opponent's cooperation rate *after our C*, its cooperation rate *after our D*, its switch rate, the fraction of the match already played, a saturating measure of how many rounds we have seen of this opponent, and a first-round flag.

The two conditional cooperation rates are the crux. A reciprocator and an unconditional cooperator have identical marginal cooperation rates when we have been cooperating; they differ only in what happens after we defect. Without those two features a single network cannot represent the policy "exploit pushovers, reciprocate with reciprocators" at all, which is exactly the policy the tournament setting demands. They are the learned counterpart of step 2 of the heuristic, and they are the reason a single policy can play a whole field. All statistics are maintained incrementally, so encoding costs $O(1)$ per round rather than $O(\text{rounds})$.

Network, Learning Rule and Training

Network. A 21–64–64–2 multilayer perceptron with ReLU hidden units, giving the Q-values of the two actions. Training uses Adam (learning rate 5×10^{-4}, gradient-norm clipping at 10) on a Huber loss with $\delta = 1$; rewards are the per-round payoffs divided by 5 to keep the value function in a comfortable range.

Learning rule. Standard DQN with experience replay (capacity $200{,}000$ transitions, batch size 64, a $2{,}000$-transition warm-up, one gradient step every two rounds played), a target network synchronised every 500 gradient steps, and discount $\gamma = 0.98$. The update uses the *double* DQN target: the online network selects the greedy next action and the target network evaluates it,

$$y = r + \gamma\,(1 - d)\,Q_{\bar{\theta}}\Big(s',\ \arg\max_{a'} Q_{\theta}(s', a')\Big)\,.$$

As argued above, in this game the overestimation bias of the plain $\max_{a'} Q_{\bar{\theta}}$ target is not neutral: defection is the highest-immediate-payoff action, so an optimistic bootstrap systematically inflates the value of defecting.

Training. One episode is one match against an opponent drawn uniformly from the eight-strategy training pool; with probability 0.2 the opponent is instead a frozen copy of the agent itself, refreshed every 200 episodes. Exploration is ε-greedy with ε decaying linearly from 1.0 to 0.05 over the first 60% of episodes. Match lengths are drawn uniformly from $[60, 140]$ rounds, and because the state contains the *fraction* of the match played rather than an absolute round counter, end-game behaviour learned on short matches transfers to the 200-round matches used at evaluation time.

The pool and the self-play share both matter for what is learned rather than how fast. Against a single fixed opponent the best reply is trivial and transfers to nothing; sampling the pool is what turns the task into "identify, then best-respond," which is what the conditional features exist to support. Self-play is what teaches the agent to sustain mutual cooperation with a strong reciprocator, which is a large share of the tournament field.

Model selection. Q-learning against a shifting opponent distribution is not monotone: late in training the agent plays mostly cooperative matches, its replay buffer fills with them, and it can forget how to punish a defector. We therefore evaluate greedily every 100 episodes and keep the best snapshot. The selection criterion uses the *training* opponents only. The three held-out opponents are played during evaluation for reporting, but never influence which checkpoint is kept, so they remain a clean measurement of generalisation. In our five runs the kept snapshot came from episodes 2800, 100, 2200, 1700 and 2100 out of 3000 — the second of these is a vivid illustration of why selection was necessary.

Each run performs about $148{,}500$ gradient steps and takes 44 seconds on a laptop CPU.

Experimental Setup

Opponent split. The eight *training* opponents are `always_cooperate`, `always_defect`, `tit_for_tat`, `suspicious_tit_for_tat`, `grim_trigger`, `pavlov`, `generous_tit_for_tat` and `random`. The three *held-out* opponents are `tit_for_two_tats`, `prober` and the `heuristic` baseline. They are never sampled during training and never enter model selection. The held-out set is deliberately not adversarial: two of the three are ordinary well-known strategies, and all three are *behaviourally*

reachable from the training pool, in the sense that an agent that had understood retaliation and forgiveness would handle them without having seen them.

Runs. Five networks are trained independently (seeds 0–4, 3,000 episodes each). The round robin is played 25 times over 200-round matches with the full twelve-agent field, rotating through the five trained networks so that the reported spread reflects training-seed variation as well as match randomness. All randomness in the evaluation is controlled by one seed.

Instruments. We report four measurements, each answering a question raised by the previous one:

1. **Round robin** — the headline standings: score per round, match win rate, cooperation rate, mean rank, tournaments won.
2. **Duels** — every focus agent against every opponent one-on-one, reported separately for training and held-out opponents. This is the generalisation measurement.
3. **Behavioural probes** — scripted situations that yield one interpretable number each: is the agent *nice* (never the first to defect against a reciprocator); its *retaliation rate* (fraction of the five rounds after a single unprovoked defection spent defecting); its score against an unconditional cooperator (3.0 means it never exploits, 5.0 that it always does) and against an unconditional defector (1.0 means it always punishes); *spiral recovery* (mutual cooperation rate in the second half of a match against an opponent that opens with five defections and then plays tit-for-tat); and its mutual cooperation rate with tit-for-tat. These turn score differences into strategy descriptions.
4. **Ablations** — double Q-learning, self-play, the long discount and the multi-opponent pool removed one at a time, over 3 seeds $\times$ 1,200 episodes. This budget is deliberately smaller than the main runs and its numbers are therefore not comparable to the main table.

Statistics. Differences between agents are reported with Welch's unequal-variance t-test (Welch 1947) and a 95% bootstrap confidence interval (Efron 1979) over the 25 per-tournament scores. Aggregation is per agent *type* across tournaments.

Results

Standings

Table 2 gives the round-robin standings. Plain tit-for-tat wins the field, the hand-written heuristic is second, and the learned agent is fourth of twelve at 2.542 points per round.

Table 3 makes the comparisons precise. The learned agent is far ahead of the random baseline it replaces, and reliably *behind* both the hand-written heuristic and plain tit-for-tat. Two remarks on the standings before we explain the gap.

First, `always_defect` has by far the highest match win rate (0.91) and the lowest score. This is the standard reminder that head-to-head wins are the wrong objective in the IPD — a point Axelrod made in 1980, and one that a reinforcement learner optimising per-round return gets right automatically.

Agent	score/rd	±sd	win	coop	rank
`tit_for_tat`	2.701	0.007	0.00	0.82	1.56
`heuristic`	2.651	0.053	0.69	0.49	2.48
`generous_tit_for_tat`	2.604	0.119	0.00	0.88	2.96
`dqn`	2.542	0.125	0.25	0.62	4.16
`tit_for_two_tats`	2.515	0.019	0.00	0.79	4.12
`susp._tit_for_tat`	2.434	0.007	0.49	0.66	6.08
`prober`	2.368	0.092	0.30	0.43	7.12
`pavlov`	2.313	0.026	0.04	0.74	7.68
`grim_trigger`	2.227	0.091	0.31	0.51	8.84
`always_cooperate`	1.953	0.136	0.00	1.00	10.80
`random`	1.925	0.087	0.39	0.50	10.84
`always_defect`	1.867	0.014	0.91	0.00	11.36

Table 2: Round-robin standings over 25 tournaments $\times$ 200 rounds. "win rate" is the fraction of matches won head to head, "coop" the cooperation rate, "rank" the mean tournament rank.

Comparison	Δ	95% CI	p
`dqn` − `random`	$+0.617$	$[+0.559, +0.675]$	1.5×10^{-23}
`heuristic` − `random`	$+0.726$	$[+0.688, +0.765]$	1.1×10^{-31}
`dqn` − `heuristic`	-0.110	$[-0.162, -0.057]$	3.1×10^{-4}
`dqn` − `tit_for_tat`	-0.160	$[-0.207, -0.111]$	1.3×10^{-6}

Table 3: Differences in score per round over the 25 per-tournament samples, with Welch t-tests and bootstrap confidence intervals.

Second, the learned agent's standard deviation (0.125) is an order of magnitude larger than tit-for-tat's (0.007) and more than twice the heuristic's (0.053). Since the DQN rotates through five independently trained networks, this spread is mostly *training-seed* variance. We return to it below, where it turns out to be the most interesting number in the table.

Where the Gap Comes From

Table 4 duels each focus agent against every opponent individually and separates training from held-out opponents.

The result is unambiguous. *On the opponents it trained against, the learned agent exactly matches the hand-written heuristic*: 2.70 versus 2.70. On the three opponents it has never seen, it scores 2.09 while the heuristic scores 2.99 and tit-for-tat scores 2.99 — a deficit of 0.90 points per round, eight times the size of the gap in the aggregate standings. Nothing about the learner's capacity or its optimisation is at fault; the entire deficit is a failure to generalise beyond the training distribution.

The individual rows show where the loss is concentrated. Against `tit_for_two_tats`, an opponent structurally similar to strategies in the pool, the network is fine (2.98). Against `prober` (1.79) and against the `heuristic` baseline (1.50) it collapses. Both of these share one property that no training opponent has: they defect *after* cooperation has been established, and they are willing to forgive afterwards. A score of 1.50 against the heuristic is barely above the

Opponent	dqn	heur.	random	tft
always_cooperate	3.79 ±1.08	4.93	4.02	3.00
always_defect	0.98 ±0.02	0.97	0.51	0.99
tit_for_tat	2.99 ±0.01	3.00	2.24	3.00
susp. tit_for_tat	2.98 ±0.01	2.98	2.22	2.50
grim_trigger	2.21 ±1.08	0.99	0.55	3.00
pavlov	3.00 ±0.00	2.83	2.37	3.00
generous_tft	2.99 ±0.01	3.01	2.77	3.00
random	2.69 ±0.20	2.90	2.30	2.23
tit_for_two_tats*	2.98 ±0.04	3.02	3.11	3.00
prober*	1.79 ±1.08	3.00	1.89	3.00
heuristic*	1.50 ±0.47	—	0.67	2.98
mean (training)	**2.70**	**2.70**	2.12	2.59
mean (held out)*	**2.09**	**2.99**	1.89	2.99
mean (all)	2.54	2.78	2.06	2.70

Table 4: Score per round in one-on-one duels, averaged over the five trained networks (standard deviations shown for the learned agent). A star marks held-out opponents.

Agent	nice	retal.	AllC	AllD	recov.	coop tft
dqn	no	0.56	3.79	0.98	**0.14**	0.99
heuristic	no	1.00	4.93	0.97	**0.99**	0.98
random	no	0.60	4.07	0.54	0.28	0.20
tit_for_tat	yes	0.20	3.00	0.99	0.00	1.00

Table 5: Behavioural probes. "recov." is the mutual cooperation rate in the second half of a match against an opponent that opens with five defections and then plays tit-for-tat. Learned-agent values are means over the five networks; "nice" is reported as true only if it holds for every seed.

mutual-defection payoff of 1.0, and the tournament head-to-head confirms the picture: the two agents score 1.502 each way, meaning they spend almost the whole match punishing each other.

Why, Behaviourally

Table 5 localises the failure to a single behaviour.

On every dimension except one the learned agent looks competent. It punishes an unconditional defector as hard as tit-for-tat does (0.98 vs 0.99), it sustains near-perfect mutual cooperation with a reciprocator (0.99), and it exploits an unconditional cooperator more than tit-for-tat does (3.79 vs 3.00). The exception is spiral recovery: 0.14, against 0.99 for the heuristic. Once the learned agent is in a stretch of mutual defection, it stays there for the rest of the match, even against an opponent that is demonstrably willing to cooperate again.

It is worth being careful about what this is. It is not a bug, and on the training distribution it is not even a mistake. From inside the state abstraction, "we are in a defection spiral with a reciprocator" and "the opponent is an unconditional defector" look nearly identical: both show a low opponent cooperation rate, low conditional cooperation after our D, and a recent window full of Ds. The training pool contains always_defect and grim_trigger, against both

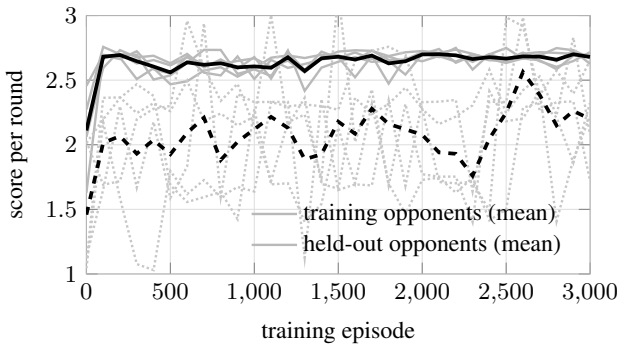


Figure 1: Greedy evaluation during training, every 100 episodes. Thin lines are the five individual seeds (solid: training opponents, dotted: held-out), thick lines their means. Performance against the training pool converges quickly and stays flat from roughly episode 500; performance against held-out opponents remains far below it and stays noisy for the whole run.

of which testing the water costs points and gains nothing. A Q-learner maximising expected return over that distribution should therefore learn exactly what it learned: never test.

The heuristic escapes precisely because its olive branch is not an expected-value calculation. It cooperates when there has been no mutual cooperation recently, full stop, and eats the cost against always_defect (0.97 — it does lose a little relative to the learner's 0.98) in exchange for recovering against everything that will forgive. Half a page of hand-written game theory beats a deep Q-network because it encodes a commitment the learner has no reason to make. This is the paper's central finding, and it is one that no aggregate score would have surfaced: on the standings the two agents are 0.110 apart, and the true difference is a single missing behaviour worth 0.90 per round on unseen opponents.

Figure 1 shows the same story during training. Averaged over seeds, score against the training pool rises from 2.11 to 2.68 and is essentially converged by episode 500. Score against the held-out opponents rises from 1.46 to 2.20 but never approaches the training-pool level and remains visibly noisy for all 3,000 episodes. There is no point in the run at which more training would have closed the gap; the generalisation deficit is present from the moment the agent has learned anything at all. A further consequence: because the checkpoint is selected on training-pool score, and the mean held-out curve peaks at 2.56 near episode 2600 while ending at 2.20, selecting on the training pool leaves held-out performance on the table — but selecting on the held-out opponents would destroy the very measurement we want to make.

Seed Variance Is Two Basins

The learned agent's standard deviations in Table 4 are strange: ±1.08 against always_cooperate, ±1.08 against grim_trigger, ±1.08 against prober, but ±0.01 against tit_for_tat. A noise process would not be that selective.

Table 6 resolves it. The five networks fall into two disjoint

	exploiting		restrained		
Seed	0	1	2	3	4
vs `always_cooperate`	4.96	4.98	3.00	3.00	3.00
vs `grim_trigger`	1.02	1.03	3.00	3.00	3.00
nice	no	no	yes	yes	yes
vs `prober`*	2.96	1.00	0.98	2.98	1.01
vs `heuristic`*	1.76	1.05	1.27	2.20	1.23
spiral recovery	0.71	0.00	0.00	0.00	0.00
mean vs training	2.665	2.742	2.691	2.711	2.710
mean vs held out*	2.573	1.653	1.753	2.727	1.745

Table 6: The five trained networks, individually. The seeds split cleanly into an exploiting and a restrained basin (top block) even though all five score within 0.08 of each other against the training pool. Held-out performance (middle and bottom blocks) varies along a *different* axis.

groups. Seeds 0 and 1 learned to *probe and exploit*: they score 4.96 and 4.98 against an unconditional cooperator — close to the maximum of 5.0 — and, having defected, are punished by `grim_trigger` for the rest of the match (1.02 and 1.03, i.e. mutual defection). Seeds 2, 3 and 4 learned to be *nice*: exactly 3.00 against `always_cooperate`, because they never defect first, and exactly 3.00 against `grim_trigger` for the same reason. There is no seed in between. The ± 1.08 standard deviations are a two-point distribution, not a spread.

The two basins score almost identically against the pool the agent trained on (2.665 and 2.742 for the exploiters, 2.691–2.711 for the restrained ones — a range of 0.08), which is why nothing in the training signal selects between them, and why model selection on training-pool score cannot either. The tournament, however, is not indifferent: an exploiter earns extra points from `always_cooperate` and loses them again to `grim_trigger`, and which of those two effects dominates depends on the composition of the field.

This has a direct methodological consequence. A paper reporting a single training run of this agent would, with probability $2/5$, describe a strategy that probes and exploits, and with probability $3/5$ a strategy that never defects first — and would in both cases report a training-pool score of roughly 2.7. These are not two samples of one strategy; they are two strategies. Multi-seed reporting in this setting is not a matter of tightening error bars, it is a matter of knowing what one has built.

Finally, note that held-out performance varies along an axis *orthogonal* to the basin split: seeds 0 and 3 handle `prober` (2.96, 2.98) while seeds 1, 2 and 4 do not (1.00, 0.98, 1.01), and this cuts across both basins. Only one network out of five — seed 0 — has any spiral recovery at all (0.71), and it is also the only exploiter that handles the held-out probing strategies. The forgiveness behaviour is thus not merely weak on average; it is absent in four of five runs and partial in the fifth.

Ablation	vs training	vs held out
full	2.692 $\pm$0.024	1.730 $\pm$0.060
`no_double_dqn`	2.717 $\pm$0.006	1.951 $\pm$0.427
`no_self_play`	2.710 $\pm$0.024	1.921 $\pm$0.452
`myopic` (γ=0.90)	2.715 $\pm$0.010	2.145 $\pm$0.276
`single_opponent`	2.527 $\pm$0.092	1.585 $\pm$0.629

Table 7: Component ablations, 3 seeds $\times$ 1,200 episodes. This budget is smaller than the main runs, so the absolute numbers are not comparable to Table 4.

Ablations

Table 7 removes one learning component at a time. Only one of them has an effect that is clearly larger than the spread across seeds: training against `tit_for_tat` alone instead of the eight-strategy pool costs 0.165 points per round against the pool, roughly seven times the full configuration's standard deviation. This confirms the design argument of the previous section — a pool, not a single opponent, is what makes the task "identify and best-respond" — and it is, notably, the only component that is about the *data* rather than the algorithm.

We report the rest honestly as null results at this budget. Double Q-learning, self-play and the long discount all leave the training-pool score within 0.03 of the full configuration, which is the same order as the seed spread. The held-out column is worse than useless here: with three seeds and standard deviations of up to 0.63 — larger than every difference in the column — it cannot distinguish any of these configurations, and readers should draw no conclusion from the fact that the ablated variants nominally score higher on it. Given the two-basin structure documented above, three seeds are simply too few: a single seed landing in one basin rather than the other moves the mean of a three-seed group by more than any of these differences. Resolving the algorithmic components properly would require the full 3,000-episode budget and considerably more seeds, which we flag as a limitation rather than a finding.

Conclusion

A standard double deep Q-network, given a 21-feature abstraction designed for exactly this task, learns a competent Iterated Prisoner's Dilemma policy: it punishes defectors, sustains mutual cooperation with reciprocators, exploits pushovers, and beats a random baseline by $+0.617$ points per round with overwhelming significance. It nevertheless loses to a hand-written heuristic that fits on half a page, and the reason is narrow and specific. Its performance on the strategies it trained against is identical to the heuristic's (2.70 vs 2.70); on strategies it has never seen it collapses (2.09 vs 2.99); and the behavioural probes name the missing capability exactly: forgiveness. With a spiral recovery of 0.14 against the heuristic's 0.99, the network cannot leave a mutual-defection spiral, because from inside its state abstraction a spiral with a reciprocator is indistinguishable from an unconditional defector, and on its training distribution never testing is optimal. The heuristic wins by mak-

ing a commitment — an unconditional olive branch — that expected-return maximisation over that distribution will not produce.

The methodological finding is the one we would most like to see adopted. Every diagnosis above came from an instrument that is not standard in this literature. The aggregate tournament table reported a respectable fourth place out of twelve and a modest 0.110-point deficit; the held-out split showed the deficit was really 0.90 and was entirely generalisation; the probes reduced it to one behaviour; and the per-seed analysis showed that what looked like variance was two distinct strategies chosen by the random seed. We therefore recommend held-out opponents, behavioural probe suites and multi-seed reporting as the default protocol for evaluating learned agents in repeated games. We are also explicit about this paper's own limitation in that light: our ablation budget of three seeds and 1,200 episodes was, by our own standard, too small to resolve the algorithmic components it studied.

Future Work

Fix forgiveness. The diagnosis points at three concrete remedies: an explicit olive-branch or optimism term in the policy, so that testing the water does not have to pay off in expectation; a recurrent or longer-window state (Hausknecht and Stone 2015) in which a spiral that *began* with our own defection is distinguishable from an opponent that defected from round one; or reward shaping that credits re-establishing mutual cooperation. The second is the most interesting scientifically, because it tests whether the failure is one of representation or of objective.

Broaden the pool. The opponent pool was the only ablation with a clear effect, so it is the axis worth scaling: larger and adversarially generated pools, curriculum ordering, and population-based training in which the pool itself evolves (Jaderberg et al. 2017; Lanctot et al. 2017; Balduzzi et al. 2019). A natural target is the Axelrod-Python strategy library (Knight et al. 2016), which would give a training pool and a held-out set of a size that makes memorisation genuinely infeasible.

Address the two basins. Ensembles over seeds, or model selection criteria that score the *behavioural profile* rather than only the return, would stop the seed from deciding which strategy a run produces. Our probe suite is already a candidate selection signal.

Harder settings. The obvious extensions along axes the framework already parameterises: noisy observations, where a move is misimplemented with some probability and forgiveness becomes essential rather than merely useful (Wu and Axelrod 1995; Ashlock, Kim, and Ashlock 2007); more than two players (Gergely 2022); unknown or stochastic match lengths, which remove the end-game exploit; and payoff matrices near the $T > R > P > S$ and $2R > T + S$ boundaries.

Stronger baselines and stronger ablations. Tabular Q-learning over the same 21-feature abstraction would price the network itself; opponent-aware learners such as LOLA (Foerster et al. 2018) and approximate tit-for-tat (Lerer and Peysakhovich 2017) are the natural strong learned baselines; and the component ablations should be rerun at full budget with enough seeds to survive the two-basin structure.

Reproducibility

The framework is pure Python 3.9+ with NumPy as the only third-party dependency. A single command, `python run_comparison.py`, trains all twenty networks (five main, fifteen ablation), plays every tournament and duel, runs every probe, and writes all numbers in this paper to `results.json`; the per-seed analysis of Table 6 and the data behind Figure 1 are produced by `python paper/analysis.py`. A full run takes a few minutes from scratch and seconds once the checkpoints are cached. Every source of randomness — network initialisation, opponent sampling, match lengths, exploration, tournament pairings and the bootstrap — is derived from an explicit seed, so all reported values are reproducible exactly.

References

Agarwal, R.; Schwarzer, M.; Castro, P. S.; Courville, A.; and Bellemare, M. G. 2021. Deep Reinforcement Learning at the Edge of the Statistical Precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*.

Ashlock, D.; and Kim, E.-Y. 2008. Fingerprinting: Visualization and Automatic Analysis of Prisoner's Dilemma Strategies. *IEEE Transactions on Evolutionary Computation*, 12(5): 647–659.

Ashlock, D.; Kim, E.-Y.; and Ashlock, W. 2007. Fingerprint analysis of the noisy prisoner's dilemma. In *IEEE Congress on Evolutionary Computation (CEC)*, 4073–4080.

Axelrod, R. 1980a. Effective Choice in the Prisoner's Dilemma. *Journal of Conflict Resolution*, 24(1): 3–25.

Axelrod, R. 1980b. More Effective Choice in the Prisoner's Dilemma. *Journal of Conflict Resolution*, 24(3): 379–403.

Axelrod, R. 1984. *The Evolution of Cooperation*. New York: Basic Books.

Axelrod, R. 1987. The Evolution of Strategies in the Iterated Prisoner's Dilemma. In Davis, L., ed., *Genetic Algorithms and Simulated Annealing*, 32–41. Morgan Kaufmann.

Axelrod, R.; and Hamilton, W. D. 1981. The Evolution of Cooperation. *Science*, 211(4489): 1390–1396.

Balduzzi, D.; Garnelo, M.; Bachrach, Y.; Czarnecki, W. M.; Pérolat, J.; Jaderberg, M.; and Graepel, T. 2019. Open-ended Learning in Symmetric Zero-sum Games. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, 434–443.

Bowling, M.; and Veloso, M. 2002. Multiagent learning using a variable learning rate. *Artificial Intelligence*, 136(2): 215–250.

Buşoniu, L.; Babuška, R.; and De Schutter, B. 2008. A Comprehensive Survey of Multiagent Reinforcement Learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, 38(2): 156–172.

Chong, S. Y.; Humble, J.; Kendall, G.; Li, J.; and Yao, X. 2007. The Iterated Prisoner's Dilemma: 20 Years On. In Kendall, G.; Yao, X.; and Chong, S. Y., eds., *The Iterated Prisoners' Dilemma: 20 Years On*, 1–21. World Scientific.

Claus, C.; and Boutilier, C. 1998. The Dynamics of Reinforcement Learning in Cooperative Multiagent Systems. In *Proceedings of the Fifteenth National Conference on Artificial Intelligence (AAAI)*, 746–752.

Cobbe, K.; Hesse, C.; Hilton, J.; and Schulman, J. 2020. Leveraging Procedural Generation to Benchmark Reinforcement Learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, 2048–2056.

Cobbe, K.; Klimov, O.; Hesse, C.; Kim, T.; and Schulman, J. 2019. Quantifying Generalization in Reinforcement Learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, 1282–1289.

Colas, C.; Sigaud, O.; and Oudeyer, P.-Y. 2019. A Hitchhiker's Guide to Statistical Comparisons of Reinforcement Learning Algorithms. arXiv:1904.06979.

Eccles, T.; Hughes, E.; Kramár, J.; Wheelwright, S.; and Leibo, J. Z. 2019. Learning Reciprocity in Complex Sequential Social Dilemmas. arXiv:1903.08082.

Efron, B. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1): 1–26.

Foerster, J. N.; Chen, R. Y.; Al-Shedivat, M.; Whiteson, S.; Abbeel, P.; and Mordatch, I. 2018. Learning with Opponent-Learning Awareness. In *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 122–130.

Fogel, D. B. 1993. Evolving Behaviors in the Iterated Prisoner's Dilemma. *Evolutionary Computation*, 1(1): 77–97.

Fudenberg, D.; and Maskin, E. 1986. The Folk Theorem in Repeated Games with Discounting or with Incomplete Information. *Econometrica*, 54(3): 533–554.

Gergely, M. 2022. Finding Cooperation in the N-Player Iterated Prisoner's Dilemma with Deep Reinforcement Learning. *Procedia Computer Science*, 207: 465–474.

Glynatsi, N. E.; and Knight, V. A. 2021. A bibliometric study of research topics, collaboration, and centrality in the iterated prisoner's dilemma. *Humanities and Social Sciences Communications*, 8: 45.

Harper, M.; Knight, V.; Jones, M.; Koutsovoulos, G.; Glynatsi, N. E.; and Campbell, O. 2017. Reinforcement learning produces dominant strategies for the Iterated Prisoner's Dilemma. *PLOS ONE*, 12(12): e0188046.

Hausknecht, M.; and Stone, P. 2015. Deep Recurrent Q-Learning for Partially Observable MDPs. In *AAAI Fall Symposium Series*.

He, H.; Boyd-Graber, J.; Kwok, K.; and Daumé III, H. 2016. Opponent Modeling in Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *PMLR*, 1804–1813.

Heinrich, J.; and Silver, D. 2016. Deep Reinforcement Learning from Self-Play in Imperfect-Information Games. arXiv:1603.01121.

Henderson, P.; Islam, R.; Bachman, P.; Pineau, J.; Precup, D.; and Meger, D. 2018. Deep Reinforcement Learning That Matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3207–3214.

Hernandez-Leal, P.; Kartal, B.; and Taylor, M. E. 2019. A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 33(6): 750–797.

Hessel, M.; Modayil, J.; van Hasselt, H.; Schaul, T.; Ostrovski, G.; Dabney, W.; Horgan, D.; Piot, B.; Azar, M.; and Silver, D. 2018. Rainbow: Combining Improvements in Deep Reinforcement Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3215–3222.

Hilbe, C.; Nowak, M. A.; and Sigmund, K. 2013. Evolution of extortion in Iterated Prisoner's Dilemma games. *Proceedings of the National Academy of Sciences*, 110(17): 6913–6918.

Hingston, P.; Dyer, D.; Barone, L.; French, T.; and Kendall, G. 2007. Opponent Modelling, Evolution, and the Iterated Prisoner's Dilemma. In *The Iterated Prisoners' Dilemma: 20 Years On*, 139–170. World Scientific.

Hingston, P.; and Kendall, G. 2004. Learning versus evolution in iterated prisoner's dilemma. In *Proceedings of the 2004 Congress on Evolutionary Computation (CEC)*, 364–372.

Huber, P. J. 1964. Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics*, 35(1): 73–101.

Hughes, E.; Leibo, J. Z.; Phillips, M. G.; Tuyls, K.; Duéñez-Guzmán, E. A.; García Castañeda, A.; Dunning, I.; Zhu, T.; McKee, K. R.; Koster, R.; Roff, H.; and Graepel, T. 2018. Inequity aversion improves cooperation in intertemporal social dilemmas. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*.

Jaderberg, M.; Dalibard, V.; Osindero, S.; Czarnecki, W. M.; Donahue, J.; Razavi, A.; Vinyals, O.; Green, T.; Dunning, I.; Simonyan, K.; Fernando, C.; and Kavukcuoglu, K. 2017. Population Based Training of Neural Networks. arXiv:1711.09846.

Jaques, N.; Lazaridou, A.; Hughes, E.; Gulcehre, C.; Ortega, P. A.; Strouse, D.; Leibo, J. Z.; and de Freitas, N. 2019. Social Influence as Intrinsic Motivation for Multi-Agent Deep Reinforcement Learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, 3040–3049.

Kingma, D. P.; and Ba, J. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*.

Kirk, R.; Zhang, A.; Grefenstette, E.; and Rocktäschel, T. 2023. A Survey of Zero-shot Generalisation in Deep Reinforcement Learning. *Journal of Artificial Intelligence Research*, 76: 201–264.

Knight, V.; Campbell, O.; Harper, M.; Gaffney, T. J.; and Glynatsi, N. E. 2025. Reviving, reproducing, and revisiting Axelrod's second tournament. arXiv:2510.15438.

Knight, V.; Campbell, O.; Harper, M.; et al. 2016. An Open Framework for the Reproducible Study of the Iterated Prisoner's Dilemma. *Journal of Open Research Software*, 4(1): 35.

Knight, V.; Harper, M.; Glynatsi, N. E.; and Gillard, J. 2024. Recognising and evaluating the effectiveness of extortion in the Iterated Prisoner's Dilemma. *PLOS ONE*, 19(7): e0304641.

Kreps, D. M.; Milgrom, P.; Roberts, J.; and Wilson, R. 1982. Rational cooperation in the finitely repeated prisoners' dilemma. *Journal of Economic Theory*, 27(2): 245–252.

Lanctot, M.; Zambaldi, V.; Gruslys, A.; Lazaridou, A.; Tuyls, K.; Pérolat, J.; Silver, D.; and Graepel, T. 2017. A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning. In *Advances in Neural Information Processing Systems 30 (NIPS)*.

Lee, C.; Harper, M.; and Fryer, D. 2015. The Art of War: Beyond Memory-one Strategies in Population Games. *PLOS ONE*, 10(3): e0120625.

Leibo, J. Z.; Zambaldi, V.; Lanctot, M.; Marecki, J.; and Graepel, T. 2017. Multi-agent Reinforcement Learning in Sequential Social Dilemmas. In *Proceedings of the 16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 464–473.

Lerer, A.; and Peysakhovich, A. 2017. Maintaining cooperation in complex social dilemmas using deep reinforcement learning. arXiv:1707.01068.

Letcher, A.; Foerster, J.; Balduzzi, D.; Rocktäschel, T.; and Whiteson, S. 2019. Stable Opponent Shaping in Differentiable Games. In *International Conference on Learning Representations (ICLR)*.

Lin, L.-J. 1992. Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching. *Machine Learning*, 8(3–4): 293–321.

Littman, M. L. 1994. Markov games as a framework for multi-agent reinforcement learning. In *Machine Learning Proceedings 1994 (ICML)*, 157–163.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. 2013. Playing Atari with Deep Reinforcement Learning. NIPS Deep Learning Workshop 2013, arXiv:1312.5602.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533.

Molander, P. 1985. The Optimal Level of Generosity in a Selfish, Uncertain Environment. *Journal of Conflict Resolution*, 29(4): 611–618.

Nowak, M. A.; and Sigmund, K. 1992. Tit for tat in heterogeneous populations. *Nature*, 355(6357): 250–253.

Nowak, M. A.; and Sigmund, K. 1993. A strategy of win-stay, lose-shift that outperforms tit-for-tat in the Prisoner's Dilemma game. *Nature*, 364(6432): 56–58.

Peysakhovich, A.; and Lerer, A. 2018. Prosocial Learning Agents Solve Generalized Stag Hunts Better than Selfish Ones. In *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2043–2044.

Press, W. H.; and Dyson, F. J. 2012. Iterated Prisoner's Dilemma contains strategies that dominate any evolutionary opponent. *Proceedings of the National Academy of Sciences*, 109(26): 10409–10413.

Sandholm, T. W.; and Crites, R. H. 1996. Multiagent reinforcement learning in the Iterated Prisoner's Dilemma. *BioSystems*, 37(1–2): 147–166.

Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D. 2016. Prioritized Experience Replay. In *International Conference on Learning Representations (ICLR)*.

Selten, R.; and Stoecker, R. 1986. End behavior in sequences of finite Prisoner's Dilemma supergames: A learning theory approach. *Journal of Economic Behavior & Organization*, 7(1): 47–70.

Stewart, A. J.; and Plotkin, J. B. 2013. From extortion to generosity, evolution in the Iterated Prisoner's Dilemma. *Proceedings of the National Academy of Sciences*, 110(38): 15348–15353.

Thrun, S.; and Schwartz, A. 1993. Issues in Using Function Approximation for Reinforcement Learning. In *Proceedings of the 1993 Connectionist Models Summer School*. Lawrence Erlbaum.

van Hasselt, H. 2010. Double Q-learning. In *Advances in Neural Information Processing Systems 23 (NIPS)*, 2613–2621.

van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep Reinforcement Learning with Double Q-Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2094–2100.

Wang, Z.; Schaul, T.; Hessel, M.; van Hasselt, H.; Lanctot, M.; and de Freitas, N. 2016a. Dueling Network Architectures for Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *PMLR*, 1995–2003.

Wang, Z.; Zhou, Y.; Lien, J. W.; Zheng, J.; and Xu, B. 2016b. Extortion can outperform generosity in the iterated prisoner's dilemma. *Nature Communications*, 7: 11125.

Watkins, C. J. C. H.; and Dayan, P. 1992. Technical Note: Q-Learning. *Machine Learning*, 8(3–4): 279–292.

Welch, B. L. 1947. The Generalization of "Student's" Problem when Several Different Population Variances are Involved. *Biometrika*, 34(1–2): 28–35.

Wu, J.; and Axelrod, R. 1995. How to Cope with Noise in the Iterated Prisoner's Dilemma. *Journal of Conflict Resolution*, 39(1): 183–189.

Zhang, C.; Vinyals, O.; Munos, R.; and Bengio, S. 2018. A Study on Overfitting in Deep Reinforcement Learning. arXiv:1804.06893.